PROOF OF CONCEPT · OPERATIONS RESEARCH

Train Unit Shunting: from a black-box solver to an observable, corrected and benchmarked optimisation pipeline

Branch-and-Price · column generation · a model flaw found by visual inspection · a pricing-based LNS



BACCAR Bedis · ESTEVE Erwann · VERBOIS Maël
Projet PORT 2026 — Master Parisien de Recherche Opérationnelle

EXECUTIVE SUMMARY

An observable, audited, physically valid solver, plus a fast primal method

1 · Observability

Instrumented the B&P (event trace: CG iterations, tree nodes, bounds, incumbents, root LP) and built a web app that replays any solution on the station map from $t = 0$ to T and every solve on its own clock.

2 · Correctness audit

The app exposed **8 solver defects**: a non-deterministic graph (out-of-bounds read), B&B nodes silently dropped, nodes solved under the wrong restrictions, a decreasing global LB, a time limit not enforced inside CG… All fixed and regression-tested.

3 · Model correction

Watching the animation revealed trains **crossing head-on on a single track**. The node-only conflict model allows it: **89 crossings** in the 10 Gurobi solutions. Fixed with exact link resources; the pricing algorithm is unchanged and the reduced costs stay exact (66 548 columns cross-checked).

4 · Large Neighbourhood Search

A second method reusing pricing and Lagrangian bounds: greedy insertion *by the pricing oracle*, price-and-dive repair, neighbourhoods pruned by their bound (91 % cut on small 1). Feasible solution in ms on every instance; matches B&P on small 1 in 11 s.

So what. The original results (gaps 0–72 %) were produced by a model that allows physically impossible plans. Report the corrected model; use B&P for bounds and LNS for fast, valid plans on large scenarios.

PROBLEM

TUSP: route, park and service each train in its window, or refuse it

Decisions for each train k with stay $[a_k, d_k]$ and required services S_k:

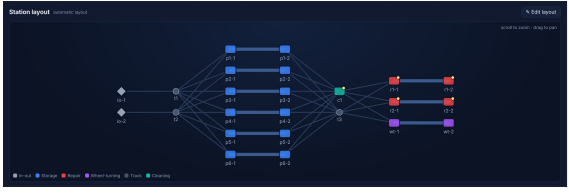
- a path through the station: arrival at an in/out node, shunting moves, services on capable nodes, storage / turning, dwelling, departure;
- or **refusal** (dummy path) at a penalty cost.

Constraints: each physical node holds at most one train per time step (incl. headway t_h); every required service performed once.

Costs (integers): shunting $w_s d + c_{shu}$, service c_s, storage $w_{dw} t_{turn} + c_{park}$, dwelling $w_{dw} t_s$, refusal $(w_s + w_{dw} + c_{park} + c_{shu})(d_k - a_k) + 2\sum_s c_s$.

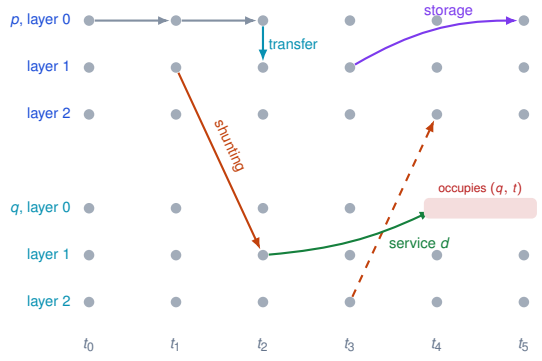
Source: instance CSV files; arc counts from the C++ graph builder ($t_s = 5$).

Station	nodes	links	trains (sc. 1–5)	time-space arcs
small	24	41	8 – 24	14 k – 41 k
medium	46	129	15 – 55	36 k – 101 k
large	74	294	25 – 105	76 k – 205 k
variant	18	19	10 – 26	13 k – 98 k



small station, automatic schematic layout (in/out → throat → 6 two-ended sidings → services)

A 3-layer time-space network turns each train's plan into a path



- **Layer 0** = standing (dwelling, end of service/storage); **layers 1, 2** = moving in each direction of a link.
- Arcs: arrival, shunting (layer l, $t \to t + d$), service & storage (layer $l \to 0$), dwelling, state transfer ($0 \to l$), departure, **dummy** (refusal).
- Each arc carries its occupation list $\mathrm{occ}(a) \subseteq \mathcal{R} \times \mathcal{T}$: the resources it blocks (node p at t; headway t_h after a departure).
- Train-specific access: storage arcs per train type, service arcs only if $s \in S_k$.

small sce 1: 14 043 arcs, of which 8 448 storage, 2 706 shunting, 770 service.

So what. Any feasible plan of one train is a source–sink path; conflicts between trains live only in $\mathrm{occ}(a)$, which is exactly what the master problem couples.

Dantzig–Wolfe: one path column per train, unit-capacity coupling rows

$$
\begin{aligned}
\min\ & \textstyle\sum_k \sum_{c\in\Omega_k} \gamma_c\, \lambda_c \\
\text{s.t.}\ & \textstyle\sum_{c\in\Omega_k} \lambda_c \ \geq 1 && (\pi_k)\ \ \forall k \\
& \textstyle\sum_{c\in\Omega_k} a_c^s\, \lambda_c \ \geq 1 && (\alpha_{ks})\ \forall k,\ s \in S_k \\
& \textstyle\sum_k \sum_{c\in\Omega_k} b_c^{rt}\, \lambda_c \ \leq 1 && (\mu_{rt} \leq 0)\ \forall (r,t) \\
& \lambda_c \in \{0,1\} \quad (\text{LP: } \lambda \geq 0)
\end{aligned}
$$

γ_c path cost, $a_c^s = 1$ if c performs s, $b_c^{rt} = |\{a \in c : (r,t) \in \mathrm{occ}(a)\}|$.

- Ω_k is exponential: columns are generated by **pricing** (next slide).
- The **dummy column** of each train (refusal) is in the RMP from the start $\Rightarrow$ every restricted master is feasible.
- Resources $\mathcal{R}$ = physical nodes (original model) $\cup$ links (corrected model, slide 13).
- Branch-and-Price on top: the LP bound is computed by column generation at every node.

So what. The coupling constraints are generic "(resource, time) $\leq$ 1" rows built from $\mathrm{occ}(a)$: adding a new kind of resource does not change the structure of the master or of the pricing.

Pricing is an RCSPP on a DAG and yields a valid bound at every iteration

Reduced cost of an arc for train k:

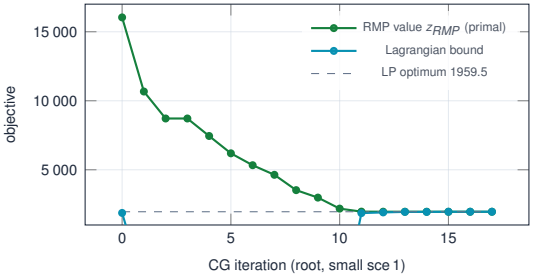
$$\bar{c}_a = c_a(k) - \sum_{(r,t)\in \mathrm{occ}(a)} \mu_{rt} \; - \; \alpha_{ks}\,[a \text{ performs } s \in S_k]$$

Labels (v, m) with $m \in \{0,1\}^{|S_k|}$ = services done; topological sweep over time; sink reached only with $m = \mathbf{1}$. Cost $O(|A|\,2^{|S_k|})$ per train. Column added if $\bar{c}(P) - \pi_k < 0$.

Lagrangian bound (valid at each CG iteration):

$$LB = z_{RMP} + \textstyle\sum_k \min\left(0,\ \bar{c}_k^*\right)$$

CG stops when $|z_{RMP} - LB| / LB \le$ gap (per depth tier, tunable in the app).



So what. The RMP starts at the all-refused value (16 040) and meets the Lagrangian bound after 18 iterations: the root LP bound 1959.5 equals the one of the original Gurobi runs, which validates our test set-up.

Source: solver trace, small station sce 1, corrected model (CLP/CBC test solver; identical root bound with Gurobi).

Branch-and-Price: two branching rules, two heuristics, every knob exposed

Branching on a fractional train k:

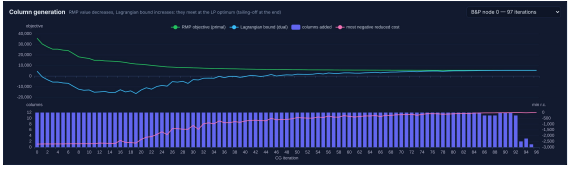
- **service split**: choose (k, s, τ) balancing the fractional flow of service s started before/after τ, divided by the time gap ⇒ left: start $\leq \tau$, right: start $> \tau$ (forbidden arcs per train);
- **quantum** (fallback): first arc where the fractional paths of k diverge; outgoing arcs split into two flow-balanced sets.

Node selection: best-bound first or DFS. **Heuristics** every N nodes: diving (fix the most integral column, re-price) and a MIP on the restricted master.

CG gap tiers: root / depth ≤ 5 / ≤ 10 / deeper.



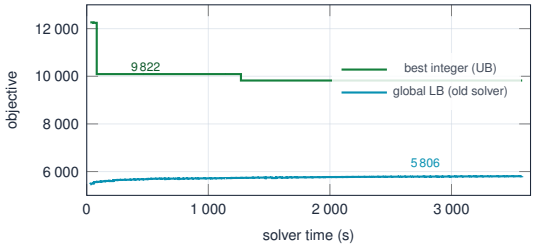
app, Search tree tab: node status replayed on the solver clock



root CG of small sce 2: 97 iterations, then tailing-off

So what. The CG on small 2 shows the classic tailing-off: 12 columns per iteration until the very end, most time spent re-solving the master.

Gurobi baseline (1 h): 26–72 % gaps, and the master LP takes the time



inst.	trains	gap %	nodes	time (s)	master %
S1	8	0	17	0.8	49
S2	12	69.2	396	3613	66
S3	16	41.7	115	3602	89
S4	18	32.9	131	3718	87
S5	24	26.5	11	3804	94
M1	15	0	13	5.7	–
M2	25	51.6	31	3697	94
M3	35	72.4	6	5911	96
M4	45	30.6	6	5074	96

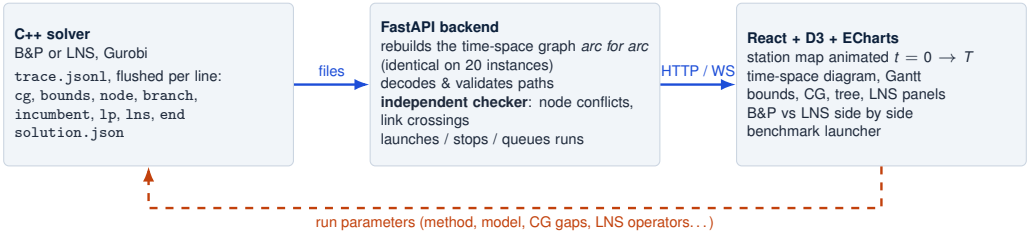
M3 ran 64 % over its 1 h limit (the deadline was not checked inside CG); M1's master time was negative (non-monotonic clock).

small sce 2 (12 trains), 396 nodes in 1 h, gap 69 %

So what. The LB barely moves in 1 h and only 6–31 nodes fit in the time on medium instances: the bottleneck is the master LP and the bound, not the search. This motivated (i) a trustworthy LB, (ii) a primal method that does not depend on the tree.

Source: `results/summary_benchmark.csv` and `dynamics_small_station_sce_2.csv` (original solver, Gurobi). LB values of the original solver are not guaranteed (see slide 11).

Observability end to end: event trace, independent decoder, replay app



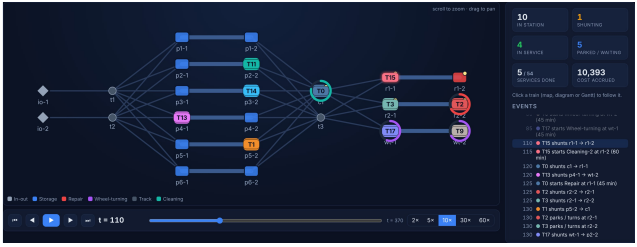
20 / 20
instances where the Python graph = the C++ graph (arcs, costs, occupations)

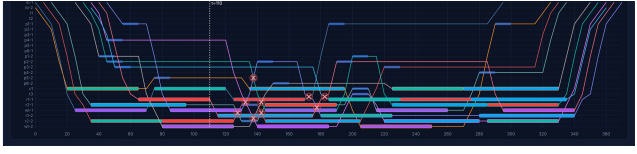
89
head-on crossings found by the checker in the 10 Gurobi solutions

live
trace streamed over WebSocket: curves and tree grow during the solve

So what. The decoder never trusts the solver: every displayed plan is re-validated from the instance data, which is what made the bugs and the model flaw visible.

Replaying a plan makes it readable: here, refusals are 84 % of the cost





small sce 4 at $t = 110$ (Gurobi solution, original model)

- tokens move along the links during shunting; rings = service progress; P = storage / turning;
- KPIs at t: 10 trains in the station, 4 in service, 5/54 services done;
- event log and time-space diagram (x = time, y = node) share the same clock; clicking any of them seeks;
- cost breakdown of the plan: **rejection 84 %**, service 8 %, shunting 5 %, dwelling 2 %.

So what. 18 trains, 8 refused: the objective (36 815) is dominated by the refusal penalty. Capacity of the service tracks, not routing, is the binding resource; this changes how the LNS chooses neighbourhoods (“refused” operator).

The audit found 8 solver defects; three invalidate published numbers

Defect	How it showed up	Consequence	Fix
Out-of-bounds read `services[n]` in the graph builder	arc count 32 470 vs 32 174 on small 4; decoder needed 3 graph variants to read old results	**graph differs run to run**	loop $s = 0..n-1$
Node fetched after a CG prune is skipped	tree in the app had unexplored children marked done	**"optimal" not proven**	single `next_node()`
Bound-pruned node does not switch state	next node solved with the previous node's forbidden arcs	**wrong LBs / prunes**	switch before prune
Global LB ignores DFS-pending nodes; child LB overwritten by a weaker Lagrangian bound	Final LB $<$ Root LB (small 3: 20 473 $<$ 20 534)	non-monotone, invalid LB	min over open nodes, max(parent, own)
DFS switch empties the stack	`--dfs` had no effect	option ignored	transfer pending nodes
Deadline not checked inside CG	436 s used with a 90 s limit (small 5); M3: 5 911 s for 1 h	time limit violated	check in CG, keep best *LB*
`high_resolution_clock`	negative master time (M1: -0.85 s)	wrong profiling	`steady_clock`
Benchmark CSV	Final_LB column contained the UB	misleading table	fixed column

So what. The three highlighted defects affect the correctness of bounds or of the search, so the original gaps are not guaranteed. All results after slide 12 come from the fixed solver.

The crossing flaw was spotted in the animation, then measured

1 · Observed in the replay



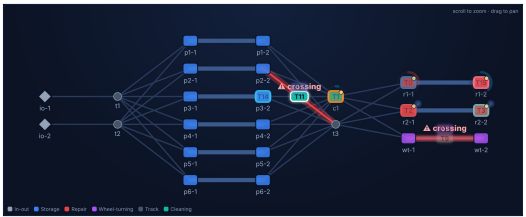
T3 and T11 on the repair track `r1-1`–`r1-2`, moving towards each other in the same step: both ends of one physical track.

2 · Why the model allows it

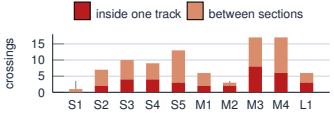
	step t	step t+5
A: $p \to q$	(p, t)	$(q, t{+}5)$
B: $q \to p$	(q, t)	$(p, t{+}5)$
link p–q	**never occupied**	

No (node, time) row is shared ⇒ the swap is feasible. A shunting arc only occupies its *departure* node at t_1 (`helper_build_shunting_arc`).

3 · Made systematic: the checker flags every crossing



Gurobi solution of small 4 at $t = 137$: two crossings in one step (`p2-2`↔`t3`, `wt-1`↔`wt-2`).



89 crossings in 10 solutions:
34 inside a single track (X-1↔ X-2),
55 between two sections.

The checker reproduces exactly these 89 (unit test).

So what. The flaw was invisible in every number (objective, gap, feasibility) and obvious in 2 seconds of animation: visual replay is a verification tool, not a presentation layer.

Three options: only exact link resources avoid side effects

	A · Keep the model ("each link has one track per direction")	**B · Route locking** (a move $p \to q$ also occupies q at t_1)	**C · Link resources** (1 per $\{p, q\}$ and time step, both directions)
New rows	0	0	$\lvert L\rvert(T/t_s{+}1)$, $\times 2.7$–5, mostly slack
Nonzeros / column	0	$+1$ per move	$+1$ per move ($+18$–$40\,\%$)
Swaps removed	none	all 89	all 89
Side effect	34 swaps happen *inside one track*: physically impossible whatever the links	also forbids **691** follow-up moves (train entering a section the other just left) $\Rightarrow$ doubles the headway	none: follow-ups and same-direction moves unchanged
Pricing / duals	–	unchanged (same occ mechanism)	unchanged (same occ mechanism)
Decision	**rejected** not defensible on sidings	**rejected** changes the model beyond the bug	**chosen** exact

$$\sum_k \sum_{c \in \Omega_k} b_c^{\ell\tau}\, \lambda_c \le 1 \qquad \forall \ell = \{p, q\} \in L,\ \forall \tau, \qquad b_c^{\ell\tau} = \lvert\{a \in c \cap A_{SH} : \mathrm{link}(a) = \ell,\ t_1(a) \le \tau < t_1(a) + d(a)\}\rvert$$

So what. Option B looked free (no new rows) but the counts measured on real solutions showed it changes the headway semantics: $8\times$ more moves forbidden than swaps. Measure side effects before choosing a "cheap" fix.

Source: counts computed by the decoder on the 10 Gurobi solutions of the original model.

Extra occupation entries: pricing unchanged, reduced costs exact

Implementation

- resource ids $|P|, \ldots, |P|+|L|-1$ appended to $\mathrm{occ}(a)$ of every shunting arc, **unordered** pair $\{p, q\}$;
- master rows sized by $|\mathcal{R}|$; arc ids unchanged $\Rightarrow$ old results still decodable;
- `--no-link-conflicts` / app checkbox restores the original model.

Why pricing stays exact. Master coefficients b_c^{rt} and pricing penalties $\sum_{(r,t)\in\mathrm{occ}(a)} \mu_{rt}$ are built from the *same list*; costs stay additive per arc and the graph stays acyclic, so the DP and the Lagrangian bound are unchanged. A path is on at most one link per step $\Rightarrow$ $b_c^{\ell\tau} \in \{0, 1\}$.

Verification

66 548
columns whose pricing reduced cost was recomputed from the master duals: 0 mismatch

20 / 20
instances: C++ and Python graphs identical incl. occupations, both models

0
node conflicts and 0 crossings in every solution of the corrected model

24 → 65
conflict resources on small station (nodes + 41 links)

reduced-cost check compiled with `-DTUSP_CHECK_REDUCED_COSTS`: $\gamma_c - \pi_k - \sum \alpha_{ks} - \sum b_c^{rt}\mu_{rt}$ vs DP value.

So what. The correction is local to the graph builder; correctness of the decomposition is guaranteed by construction and checked at run time.

The correction is free at the root and costly in the tree

inst.	root LB	original model UB	original model cross.	original model status	corrected model UB	corrected model cross.	corrected model status
S1	1959.5	1 965	2	optimal 3.7 s	1 974	0	gap 0.46 %
S2	5438.2	12 259	5	time limit	12 504	0	time limit
S3	20534.2	39 354	7	time limit	39 340	0	time limit
S4	27762.2	40 526	5	time limit	40 257	0	time limit

Root LB **identical** in both models on every instance (and equal to the Gurobi root LB): crossings appear only in integer solutions.

small sce 1, corrected model, 600 s

UB 1 974, LB 1 965 (gap 0.46 %) after 225 nodes, **595 s of 600 s spent in primal heuristics** (restricted-master MIP every 5 nodes).

The tuning panel of the app (heuristic frequency, MIP time) is the lever: the tree itself is small.

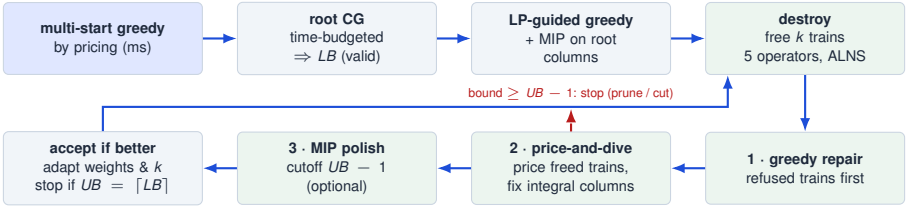
reading

The original 1 965 was only reachable with 2 swaps; the corrected optimum lies in $[1\,965,\ 1\,974]$.

So what. Report the corrected model. Its LP bound is unchanged, so the weakness of the bound (slide 8) remains the main lever; the extra cost is in branching and heuristics.

Source: CLP/CBC test solver, 90 s per run (S1 corrected: 600 s). Absolute times to be re-measured with Gurobi.

LNS: destroy a few trains, repair with the pricing, prune with the relaxation



operator	frees
random	k random trains
time	trains present with a seed train
related	trains sharing nodes and time with a seed
dual	high reduced cost under root duals
refused	refused trains + their competitors

- weights $w \leftarrow 0.8w + 0.2\,\sigma$, $\sigma = 6$ new best / 1 repaired / 0.5 MIP timeout / 0.3 pruned;
- k/n grows $\times 1.15$ when pruned or stuck, shrinks $\times 0.85$ on MIP timeout, restarts after 12 failures at the max;
- same trace format as B&P $\Rightarrow$ same app views + an LNS panel.

So what. Every component reuses the B&P machinery (RMP, pricing, Lagrangian bound): LNS is a different search over the same decomposition, not a separate model.

The pricing builds plans; the bound discards 91 % of neighbourhoods

Greedy insertion = one pricing call per train

Insert trains in a given order; for train k set $\pi_k = \gamma_{\text{dummy}(k)}$, $\alpha = 0$, $\mu_{rt} = -\text{BIG}$ on resources already used.

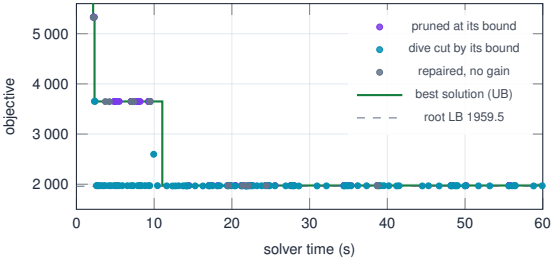
$\Rightarrow$ the DP returns the cheapest **conflict-free** path doing all S_k, or nothing (refusal cheaper). Feasible plan in $\approx n$ DP sweeps, no MIP.

Neighbourhood bound

Fixed trains keep one column, already in the RMP $\Rightarrow \bar{c}_k^* \geq 0$:

$$LB_N = z_{RMP} + \sum_{k \in F} \min(0, \bar{c}_k^*)$$

is valid for the restricted problem. Integer costs: if $LB_N \geq UB - 1$ the neighbourhood (or the dive step) cannot improve $\Rightarrow$ stop.

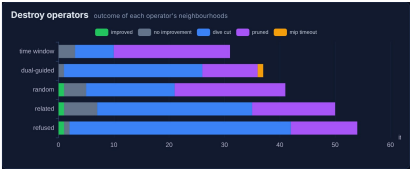


small sce 1, corrected model, 60 s: **213** neighbourhoods, **78** pruned + **116** dive cuts (91 %), 0.27 s each; UB 8 976 (greedy, 0 s) $\rightarrow$ 5 331 (root MIP) $\rightarrow$ 3 650 $\rightarrow$ **1 974 at 11 s** = B&P's best after 600 s.

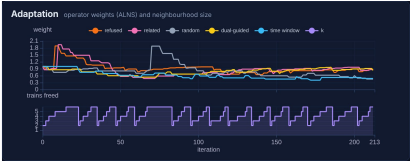
So what. The relaxation turns most iterations into a cheap proof that "no better plan exists here", so time goes to the neighbourhoods that can still improve. (Trace, CLP/CBC test solver.)

The app revealed four failure modes of the first LNS, each fixed by a rule

Observed	Cause	Rule added
RMP stuck at 101 424 for 500 CG iterations (small 5)	primal degeneracy of restricted masters	stop after 25 iterations without RMP progress (bound kept)
bound $\approx$ 1 960 $\ll$ *UB* = 3 650 yet every dive cut	dive rounded a train's *refusal* column	never round a refusal; fix only columns $\geq$ 0.99 (conflict-free)
MIP on neighbourhood columns never inserts the refused train	needed columns never priced by the LP	greedy repair inserts refused trains *first*
0 iterations in 90 s (small 4)	root CG + root MIP used the budget	root CG $\leq$ 30 % of time, root MIP 10 s



outcomes per destroy operator (small 1)



ALNS weights and neighbourhood size k over iterations

So what. With the refusal rule and refused-first insertion, small 1 went from stuck at 3 650 to 1 974 in 11 s. The trace made each failure diagnosable in minutes.

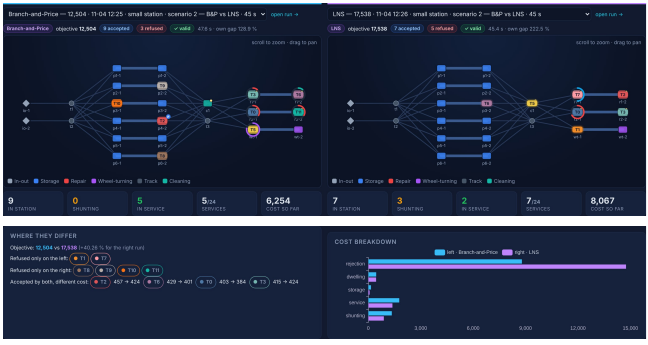
B&P wins when it has time; LNS always has a valid plan in milliseconds

	B&P		LNS	
inst.	UB	1st sol.	UB	1st sol.
S1	**1 974**	3.7 s	**1 974**	0.00 s
S2	**12 504**	47.9 s	17 538	0.01 s
S3	–	–	42 970	0.03 s
S4	–	–	53 672	0.05 s
S5	–	–	101 405	0.17 s

corrected model, 60 s per run, all plans re-checked: 0 conflicts, 0 crossings. “—”: no integer solution within the limit.

side by side (small 2, $t = 120$)

B&P refuses T1, T4, T7; LNS refuses T0, T5, T8, T11. Same instance, disjoint refusal sets: the paired schedules show where each method spends service capacity.



So what. Complementary roles: B&P for bounds and quality on small instances, LNS for fast feasible plans where the tree does not produce one. Combining them (LNS incumbent as B&P cutoff) is the natural next step.

Source: app benchmark launcher, CLP/CBC test solver; to be re-run with Gurobi.

What is proven, what still needs Gurobi, and the next four steps

Proven in this POC

- graph, costs and occupations identical C++ ↔ decoder (20 instances, both models);
- reduced costs exact (66 548 columns), Lagrangian and neighbourhood bounds valid by construction;
- every plan of the corrected model conflict- and crossing-free;
- test set-up reproduces the Gurobi root LBs exactly (1959.5, 5438.2, 20534.2, 27762.2, 63822.9).

Not yet measured

Absolute times and B&P vs LNS rankings: the CLP/CBC test solver is much slower than Gurobi on the many small re-solves of the LNS and overruns MIP time limits.

Next steps

1. **Gurobi benchmark**: small + medium × {B&P, LNS} × corrected model, 600 s, 3 seeds, via the app launcher (≈ 1 night).
2. **Dual stabilisation** (Wentges smoothing) for the degenerate RMPs that dominate master time.
3. **Hybrid**: LNS incumbent as B&P cutoff from second 1; LNS as a primal heuristic in the tree.
4. **Stronger bound**: the LP bound stays far from the UB on scenarios with many refusals (S2: 5 438 vs 12 504); study capacity cuts on service tracks.

So what. The pipeline is now trustworthy and inspectable; the remaining questions are performance questions, and the tooling to answer them (launcher, traces, side-by-side replay) is in place.